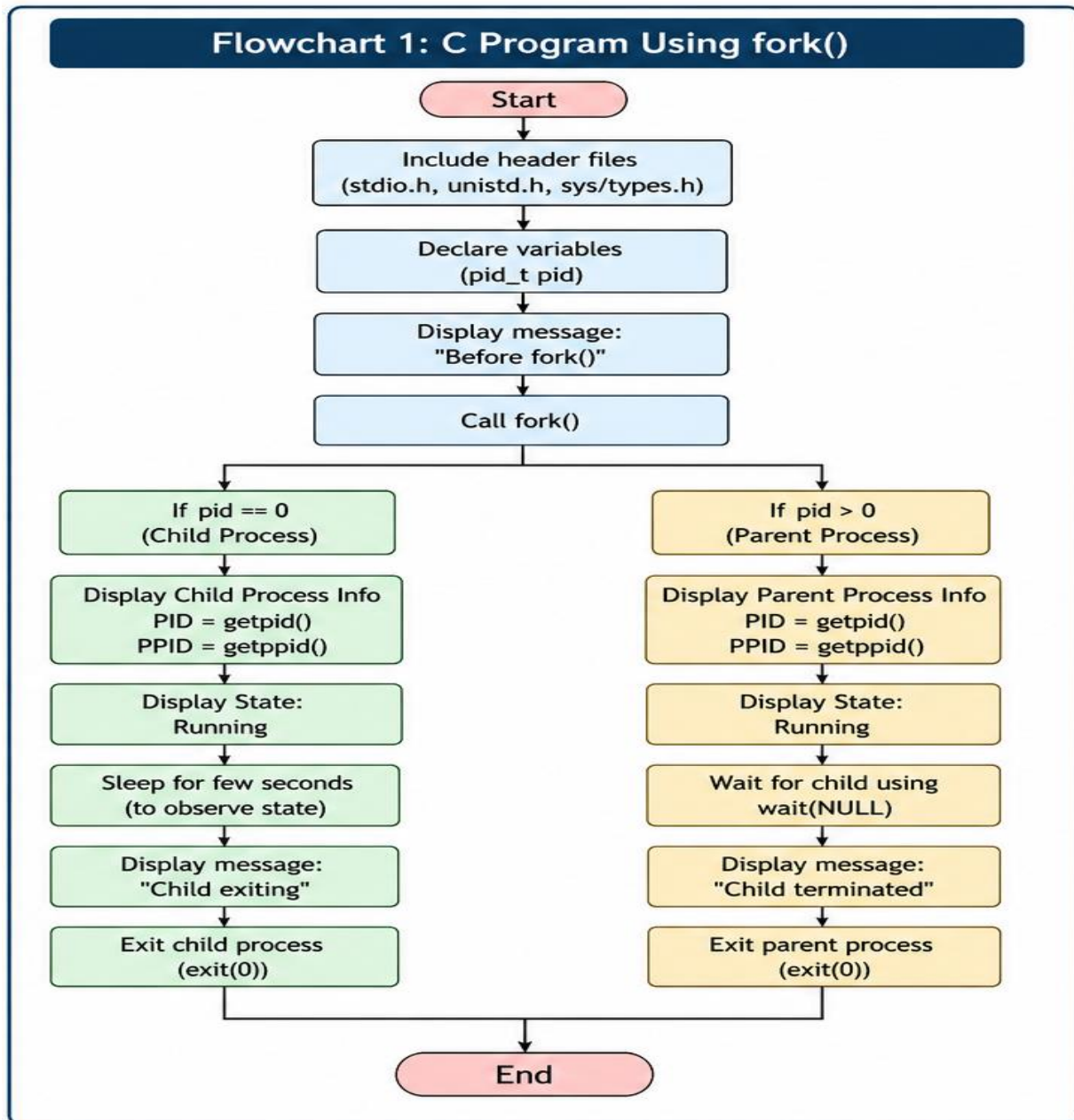


Task 3: Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Flow Chart



```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano task3_fork.c
alekhya@Alekhyareddy:~/OSSP$ gcc task3_fork.c -o task3_fork
alekhya@Alekhyareddy:~/OSSP$ ./task3_fork
Before fork()

--- Parent Process ---
Parent PID : 424
Parent PPID : 300
Parent State: Running

--- Child Process ---
Child PID : 425
Child PPID : 424
Child State: Running
Child exiting
Child terminated
Parent exiting
alekhya@Alekhyareddy:~/OSSP$ |
```

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    printf("Before fork()\n");
    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    /* Child Process */
    if (pid == 0)
    {
        printf("\n--- Child Process ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child State: Running\n");

        sleep(3);

        printf("Child exiting\n");
        exit(0);
    }

    /* Parent Process */
    else
    {
        printf("\n--- Parent Process ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Parent PPID : %d\n", getppid());
        printf("Parent State: Running\n");

        wait(NULL);

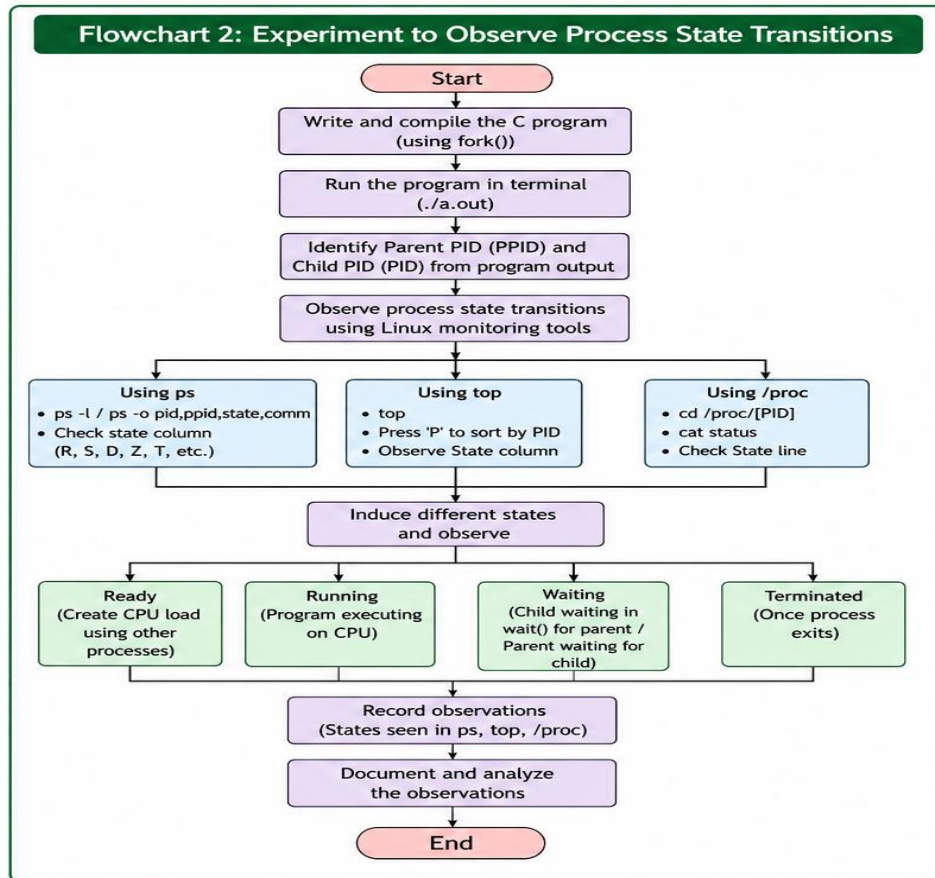
        printf("Child terminated\n");
        printf("Parent exiting\n");

        exit(0);
    }

    return 0;
}
```

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Flow Chart:



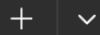
```
alekhya@AlekhyaReddy: ~/OS x + v
alekhya@AlekhyaReddy:~$ cd ~/OSSP
alekhya@AlekhyaReddy:~/OSSP$ nano task4_states.c
alekhya@AlekhyaReddy:~/OSSP$ nano task4_states.c
alekhya@AlekhyaReddy:~/OSSP$ gcc task4_states.c -o task4_states
alekhya@AlekhyaReddy:~/OSSP$ ./task4_states
Before fork()

Parent Process
Parent PID : 447
Child PID  : 448
Parent State: Running
Parent waiting for child...

Child Process
Child PID   : 448
Child PPID  : 447
Child State: Running
Child entering Waiting state...
Child exiting...
Child terminated
Parent exiting
alekhya@AlekhyaReddy:~/OSSP$ |
```



alekhya@Alekhyareddy: ~/OS



GNU nano 7.2

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork()\n");

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child State: Running\n");

        printf("Child entering Waiting state...\n");
        sleep(15);

        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        printf("Parent State: Running\n");

        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child terminated\n");
        printf("Parent exiting\n");
    }

    return 0;
}
```